

Comparação do Impacto do Custo de Identificação de Pontes para Encontrar Caminhos Eulerianos em Grafos

Adriano Araújo Domingos dos Santos¹

¹Instituto de Ciências Exatas e Informática – Pontifícia Universidade Católica de Minas Gerais (PUC Minas)

R. Cláudio Manoel, 1162 – Savassi - 30.140-100 – Belo Horizonte – MG – Brazil

{adriano.domingos}@sga.pucminas.br

Abstract. *Fleury’s algorithm constructs Eulerian paths by iteratively selecting edges, requiring bridge identification at each step to prevent premature graph disconnection. This paper presents an empirical analysis of the computational impact of different bridge detection strategies integrated into Fleury’s algorithm. This study compares the classic Tarjan’s Method (DFS-based) against Naive approaches optimized with the Union-Find data structure (Global and Local evaluations), while also assessing the impact of Adjacency List and Forward Star memory representations. Experiments conducted on sparse graphs with up to 100,000 vertices demonstrate that, although Tarjan’s algorithm is asymptotically superior for global bridge mapping, integrating a surgical, on-demand verification (Local Naive) significantly outperforms the DFS method in practice, processing up to 160.6% more edges in massive graphs. Furthermore, the static Forward Star representation proved essential to mitigate garbage collection overhead during intensive read iterations. We conclude that isolated, short-circuit connectivity validations are far more efficient than full topological re-explorations for constructing large-scale Eulerian paths.*

Resumo. *O algoritmo de Fleury constrói caminhos eulerianos por meio da seleção iterativa de arestas, exigindo a identificação de pontes a cada passo para evitar a desconexão prematura do grafo. Este trabalho apresenta uma análise empírica do impacto computacional de diferentes estratégias de detecção de pontes integradas ao algoritmo de Fleury. Foram comparados o clássico Método de Tarjan (baseado em DFS) e abordagens do método Naive otimizadas com a estrutura Union-Find (avaliações Global e Local), avaliando-se também o impacto das representações em memória de Lista de Adjacência e Forward Star. Experimentos conduzidos em grafos com até 100.000 vértices demonstram que, embora o algoritmo de Tarjan seja assintoticamente superior para o mapeamento global de pontes, a integração de uma verificação cirúrgica e sob demanda (Naive Local) supera o método DFS na prática, processando até 160,6% mais arestas em grafos massivos. Ademais, a representação Forward Star mostrou-se essencial para mitigar a sobrecarga de memória (Garbage Collector) em varreduras intensas. Conclui-se que validações isoladas de conectividade com encerramento precoce são substancialmente mais eficientes que reexplorações topológicas completas para a montagem de caminhos eulerianos em larga escala.*

1. Introdução

Um grafo $G = (V, E)$ não-direcionado é composto por um conjunto de vértices V e arestas E . Um grafo é dito conexo quando todos os vértices de um grafo conseguem alcançar todos os outros. Um componente $H = (V_H, E_H)$ é um subconjunto de vértices e arestas que pertencem a um grafo: $V(H) \subseteq V(G)$ e $E(H) \subseteq E(G)$ de modo que todas as arestas $\{v, w\}$ sejam de vértices pertencentes ao subconjunto $V(G)$.

Um caminho é uma sequência alternante de vértices e arestas de um grafo, e um ciclo é um caminho que inicia e termina no mesmo vértice. Um caminho euleriano é quando todas as arestas de um grafo são visitadas exatamente uma vez. Um grafo é dito euleriano quando houver algum ciclo euleriano válido, e é dito semi-euleriano quando existir um caminho euleriano, de modo que todo grafo euleriano também é semi-euleriano.

Uma ponte é uma aresta que ao ser removida, aumenta o número de componentes conexos de um grafo. Saber se uma aresta é uma ponte é particularmente útil para encontrar um caminho euleriano, porque se ela só poderá ser adicionada ao caminho quando for a última aresta ainda não explorada que inclui aquele determinado vértice.

2. Algoritmos

Para encontrar um caminho ou ciclo euleriano em um grafo pode-se utilizar o **método Fleury**, que requer a identificação de pontes durante sua execução, e para isso serão comparados o desempenho de dois métodos **Naive** e o **Tarjan**.

2.1. Busca em Profundidade

O algoritmo de Busca em Profundidade (DFS) consiste em explorar todos os vértices e arestas de um grafo $G = (V, E)$, de modo que ao visitar um vértice $v \in V(G)$, é escolhido um de seus vizinhos $w \in \Gamma(v)$ para ser explorado, e assim por diante, de modo que o restante da vizinhança de v só é explorada após os vértices alcançáveis $\hat{\Gamma}(w)$ por w já tiverem sido explorados [Tarjan 1972]. Esta algoritmo é essencial para vários algoritmos derivados, incluindo a identificação de pontes em grafos pelo método de Tarjan.

2.2. Método Tarjan

O método de Tarjan é um algoritmo eficiente para encontrar todas as pontes em um grafo não-direcionado em uma única execução da DFS. A ideia central é utilizar o conceito de menor vértice alcançável, que para cada vértice v , calcula-se o menor tempo de descoberta alcançável a partir de v através de zero ou mais arestas da árvore de busca, seguido de no máximo uma aresta de retorno [Tarjan 1974].

2.3. Union-Find

A estrutura de dados *Union-Find* (também conhecida como Conjuntos Disjuntos) gerencia uma coleção de elementos particionados em uma série de subconjuntos não sobrepostos. Sua utilidade principal reside na alta eficiência em determinar se dois elementos pertencem ao mesmo conjunto e em mesclar conjuntos distintos. A estrutura suporta duas operações fundamentais: o `find`, que localiza a raiz representativa do conjunto ao qual um vértice pertence, e o `union`, que conecta dois subconjuntos.

A implementação adotada faz uso de duas otimizações cruciais para alcançar desempenho quase constante. A primeira é a união por posto (*union by rank*), que, durante

a fusão, anexa a raiz da árvore mais rasa à raiz da árvore mais profunda, mantendo a altura da árvore resultante a menor possível. A segunda é a compressão de caminho (*path compression*), executada durante o `find`, que atualiza os nós visitados para apontarem diretamente para a raiz, achatando a estrutura para consultas futuras. Juntas, essas heurísticas reduzem o tempo amortizado por operação para $\mathcal{O}(V)$.

Para facilitar a identificação de pontes, a estrutura foi construída para manter um contador do número de subconjuntos de forma ativa, decrementando a contagem de componentes do grafo original (iniciado em $|V|$) a cada operação de `union` bem-sucedida. Isso permite a consulta da quantidade de componentes conexos no grafo em tempo $\mathcal{O}(1)$.

2.4. Método Naive

O método *Naive* é uma abordagem ingênua para encontrar pontes de um grafo que, para cada aresta do grafo, verifica se sua remoção aumenta o número de componentes conexos. Tradicionalmente, essa verificação exigiria remover fisicamente a aresta e executar uma DFS para recontar os componentes, gerando alto custo com as mutações da estrutura.

A implementação utilizada durante a coleta dos dados adota uma integração com a estrutura ***Union-Find*** no lugar da DFS para otimizar o processo de verificação de conectividade. Para testar se uma aresta $\{v, w\}$ é uma ponte, o algoritmo inicializa uma instância de *Union-Find* contendo todos os $v \in V(G)$ vértices desconexos. Em seguida, itera-se sobre todas as arestas de E . Caso a aresta inspecionada não seja a aresta em teste $\{v, w\}$, realiza-se a operação de `union`. Ao final da iteração, compara-se o número final de componentes computados pelo *Union-Find* com o número original de componentes do grafo intacto. Se houver divergência, a aresta inspecionada é uma ponte.

Essa heurística fez com que o grafo permanecesse inalterado durante a toda execução, permitindo que a utilização de paralelismo para analisar concorrentemente se cada aresta é ponte.

Como consequência direta do uso de *Union-Find*, a complexidade para verificar uma única aresta é de $\mathcal{O}(E)$. Para identificar todas as pontes de um grafo, o custo é de $\mathcal{O}(E^2)$ no pior caso. Embora a complexidade temporal seja quadrática em relação às arestas, essa heurística garante que o grafo original permaneça inalterado (somente-leitura) durante toda a execução. Essa ausência de mutação viabiliza a utilização de paralelismo maciço, permitindo que a checagem de múltiplas arestas concorra perfeitamente sem a necessidade de sincronismos ou bloqueios.

2.4.1. Estratégias de Execução: Global e Local

Para a integração prática com o algoritmo de Fleury durante os experimentos, o método *Naive* foi subdividido e testado em duas variações metodológicas distintas:

- ***Naive Global (Completo)***: A cada iteração do algoritmo de Fleury, o método é executado sobre todo o conjunto de arestas restantes para recalcular rigorosamente todas as pontes do grafo simultaneamente. Esta abordagem possui um custo iterativo de $\mathcal{O}(E^2)$ e foi implementada puramente para estabelecer uma base de comparação justa e estrutural com o método de Tarjan, uma vez que este, por sua

natureza atrelada à DFS, mapeia obrigatoriamente todas as pontes do grafo a cada chamada.

- **Naive Local (Sob Demanda):** Visando viabilizar a utilização do algoritmo em grafos massivos e reduzir a complexidade assintótica global, esta variação explora a vantagem isolada da verificação de conectividade em $\mathcal{O}(E)$. A cada passo da execução do Fleury, o método testa de forma independente apenas se a aresta adjacente candidata w é uma ponte. Caso a restrição seja satisfeita (a aresta não é ponte ou é a única opção), a verificação do restante do grafo é abortada e o algoritmo avança imediatamente, poupando a alocação massiva de recursos computacionais.

Essa subdivisão garante que a análise de resultados contemple tanto o pareamento exato de responsabilidades teóricas entre os algoritmos (Global vs. Tarjan) quanto o cenário de otimização onde o método ingênuo é levado ao seu limite de eficiência operacional (Local).

2.5. Método Fleury

O método de Fleury serve para encontrar um caminho ou ciclo euleriano em um grafo conexo se houver algum. Para isso, primeiro é verificadas as condições de existência, sendo elas:

- **Ciclo Euleriano:** Os graus $d(v)$ de todos os vértices $v \in V(G)$ é par.
- **Caminho Euleriano:** Os graus $d(v)$ de todos os vértices $v \in V(G)$ é par, com exceção de dois vértices.

Seja v o vértice inicial, caso exista um caminho euleriano mas não um ciclo, o vértice v deverá ser um dos vértices de grau ímpar. E a cada iteração, é escolhido um vértice w adjacente a v : $w \in \Gamma(v)$, desde que ele não seja uma ponte, salvo o caso em que w seja o único vizinho de v . Então o vértice v , a aresta $\{v, w\}$ e o vértice w são incluídos no caminho. Então o algoritmo trata v como sendo o w . Esse processo é repetido até o ponto em que não exista mais nenhum vértice na adjacência de v . E portanto sabemos se o caminho é um ciclo se o primeiro vértice for igual ao último.

3. Gerador de Grafos Não-Direcionados

Para avaliar e comparar o desempenho dos algoritmos propostos, foi desenvolvido um gerador de grafos sintéticos capaz de criar grafos conexos, eulerianos e semi-eulerianos com número de vértices, arestas ou densidades controlados.

A geração de grafos com propriedades específicas utilizam os seguintes métodos construtivos para garantir a sua conformidade:

- **Grafos Conexos:** O algoritmo utiliza o embaralhamento Fisher-Yates para distribuir os identificadores dos vértices de forma aleatória, e constrói um caminho passando por todos eles seguindo a ordem gerada, garantindo que o grafo seja conexo. Em seguida, arestas aleatórias são adicionadas iterativamente até atingir a densidade alvo ou número limite de arestas.
- **Grafos Eulerianos:** Para garantir que todo vértice possua grau par, o gerador constrói inicialmente um ciclo hamiltoniano (um ciclo longo que visita todos os vértices e retorna ao ponto de partida) utilizando a mesma abordagem utilizada

para grafos conexos. Para preencher o grafo com mais arestas sem quebrar a restrição matemática dos graus, o gerador insere novas arestas exclusivamente formando ciclos fechados, mais especificamente, triângulos (ciclos de tamanho 3).

- **Grafos Semi-Eulerianos:** Para assegurar que exatamente dois vértices possuam grau ímpar, a geração inicia-se com um caminho hamiltoniano (não fechado). De modo que os vértices das extremidades iniciam com grau 1 (ímpar), enquanto os vértices internos iniciam com grau 2 (par). O preenchimento de arestas para atingir a densidade desejada ocorre da mesma forma que nos grafos eulerianos, através da adição de ciclos de tamanho 3.

Para viabilizar a geração de grafos de grande porte com baixo consumo de memória principal, foi adotada uma técnica de empacotamento de bits. Cada aresta não-direcionada, é formada pelos vértices v e w , e cada um deles é representado por um número inteiro de 32 bits (`int`), e cada aresta é representada de forma normalizada de modo que o menor vértice ocupe os 32 bits mais significativos e o maior ocupe os 32 bits menos significativos de um tipo primitivo de 64 bits (`long`). Isso elimina a necessidade de instanciar múltiplos objetos para representar cada aresta, permitindo a validação de unicidade de arestas em tempo $\mathcal{O}(1)$ através de um conjunto *hash* de tipos primitivos.

3.1. Fundamentação Teórica da Geração por Ciclos

A estratégia de construir grafos eulerianos através da inserção iterativa de ciclos é fundamentada diretamente na teoria clássica dos grafos, unindo os trabalhos de Euler e Veblen.

O Teorema de Euler (1736) estabelece que um grafo conexo possui um ciclo euleriano se, e somente se, todos os seus vértices possuírem grau par. Consequentemente, para a existência de um caminho euleriano (grafo semi-euleriano), exige-se que exatamente dois vértices possuam grau ímpar. [Biggs et al. 1976]

Para aumentar a densidade do grafo mantendo rigorosamente essas propriedades de paridade, o algoritmo baseia-se no Teorema de Veblen, que demonstra que o conjunto de arestas de um grafo pode ser particionado em uma coleção de ciclos se, e somente se, todos os vértices do grafo tiverem grau par. [Veblen 1912]

Ao inserir um ciclo mínimo (um triângulo) aleatoriamente no grafo, o grau de três vértices distintos é incrementado em exatamente 2 unidades (uma aresta de entrada e uma de saída). Somar um número par ao grau de qualquer vértice preserva a sua paridade intrínseca. Desse modo, o algoritmo garante matematicamente a topologia correta para os grafos eulerianos e semi-eulerianos, independentemente da quantidade massiva de arestas adicionadas ao final.

4. Representações

Durante a execução dos algoritmos o grafo precisa ser representado na memória principal, para isso foram utilizadas duas abordagens que serão descritas a seguir.

4.1. Forward Star

A representação *Forward Star* é uma estrutura de dados baseada em vetores contíguos, amplamente conhecida por sua eficiência de memória e excelente localidade de cache.

Diferente das tradicionais listas de adjacência baseadas em ponteiros e objetos alocados dinamicamente, o *Forward Star* serializa a topologia do grafo em apenas dois vetores primitivos unidimensionais: `pointers` e `targets`.

Na implementação desenvolvida, a construção do grafo foi estritamente desacoplada de sua representação final através do padrão de projeto *Builder*. Durante a fase de construção, as arestas são inseridas em vetores temporários paralelos. Ao finalizar a instanciação, o construtor ordena as arestas por vértice de origem, compacta os destinos no vetor final `targets` de tamanho $|E|$, e preenche o vetor `pointers` de tamanho $|V| + 1$. Cada índice v em `pointers` armazena o índice inicial de seus vizinhos no vetor `targets`, permitindo que a varredura da vizinhança de qualquer vértice ou o cálculo do seu grau sejam realizados em tempo $\mathcal{O}(1)$.

4.1.1. O Impacto da Heurística de Imutabilidade

A principal característica arquitetural desta implementação do *Forward Star* é a sua imutabilidade. Uma vez que o grafo é compilado pelo *Builder*, os vetores `pointers` e `targets` tornam-se fixos na memória (*read-only*).

Essa decisão de design garante altíssima performance para consultas e iterações iterativas, mas torna operações de mutação topológica extremamente custosas. Para permitir sua mutabilidade, a operação de remover uma aresta exige a iteração completa de todas as arestas existentes (omitindo a aresta alvo), e a reordenação e alocação de novos vetores para criar uma nova instância do grafo.

É exatamente neste gargalo estrutural que a heurística de imutabilidade do Método Naive (descrita na Seção 2.4) demonstra o seu valor prático. Se a identificação de pontes utilizasse a abordagem convencional (remover fisicamente a aresta), executar a DFS e readicionar a aresta, cada verificação desencadearia duas reconstruções completas do grafo. O custo para testar todas as arestas saltaria de forma impraticável, além de gerar uma sobrecarga massiva no coletor de lixo devido à realocação constante de vetores.

Ao integrar o *Union-Find*, o algoritmo iterador ignora logicamente a aresta testada sem solicitar sua remoção estrutural. Isso preserva a integridade da instância estática do *Forward Star*, aproveitando ao máximo a sua velocidade de varredura sequencial em memória contígua e viabilizando a execução eficiente do método *Naive* com paralelismo.

4.2. Lista de Adjacência

A representação por Lista de Adjacência implementada neste trabalho foca na dinamicidade estrutural. A topologia do grafo é armazenada através de uma tabela de espalhamento (`HashMap`), onde a chave representa o identificador do vértice $v \in V$ e o valor é um conjunto de espalhamento genérico (`HashSet`) contendo todos os vizinhos $w \in \Gamma(v)$.

Esta estrutura baseada em coleções de objetos confere uma vantagem considerável para mutações no grafo em tempo de execução. As operações de inserção ou remoção de arestas ocorrem em tempo amortizado $\mathcal{O}(1)$. Ao contrário do *Forward Star*, que exige a recriação completa do grafo, a Lista de Adjacência permite a edição isolada da vizinhança de um vértice sem perturbar o restante da estrutura de dados. Consequentemente, para

algoritmos que dependem intrinsecamente de manipulação estrutural severa (como a tradicional busca de pontes por destruição e reconstrução), essa representação apresenta comportamento computacional significativamente superior.

Entretanto, esse modelo dinâmico carrega fortes desvantagens de desempenho em cenários de acesso massivo. A utilização de estruturas baseadas em objetos na linguagem Java (`Integer`, `HashSet`) implica em altos custos de alocação de memória e extrema fragmentação na *heap*. Como os dados não estão organizados de forma sequencial, ocorre a perda da localidade de referência do cache do processador (*cache misses*).

5. Metodologia

Para avaliar quantitativamente o impacto do custo de identificação de pontes no tempo de execução do algoritmo de Fleury, foi elaborado um experimento computacional (*benchmark*). O objetivo central é isolar e comparar o desempenho estrutural e algorítmico das combinações entre os métodos de detecção de pontes (Naive e Tarjan) e as representações em memória (*Forward Star* e Lista de Adjacência).

5.1. Design do Experimento e Parâmetros

O experimento foi desenhado para analisar grafos de diferentes magnitudes e topologias. Para garantir a significância estatística e mitigar anomalias causadas por processos de *background* do sistema operacional ou pelo *Garbage Collector* da Máquina Virtual Java (JVM), cada cenário único foi executado em 10 repetições independentes. Os parâmetros independentes (*inputs*) variados durante o experimento foram:

- **Tamanho do Grafo ($|V|$ e $|E|$):** Foram gerados grafos esparsos em quatro escalas de magnitude, variando de centenas a centenas de milhares de elementos. Os pares de vértices (n) e arestas (m) utilizados foram: (100, 250), (1.000, 2.500), (10.000, 25.000) e (100.000, 250.000).
- **Topologia (Conectividade):** Grafos Eulerianos, Semi-Eulerianos e Não-Eulerianos (grafos desconexos ou com paridade de graus inválida, incluídos como grupo de controle e validação de falha rápida).
- **Algoritmo de Identificação de Pontes:** Foram avaliadas três estratégias de execução distintas: o Método de Tarjan (avaliação integral baseada em DFS), o Método *Naive* Global (avaliação integral baseada em *Union-Find*) e o Método *Naive* Local (avaliação isolada sob demanda), conforme detalhado na Seção 2.4.1.
- **Representação Computacional:** Lista de Adjacência e *Forward Star*.

A combinação ortogonal de todas essas variáveis (4 tamanhos $\times$ 3 topologias $\times$ 3 algoritmos $\times$ 2 representações) executada 10 vezes resultou em um espaço amostral de 720 execuções documentadas.

5.2. Fluxo de Execução e Coleta de Dados

Para cada uma das 720 execuções, o ambiente de testes seguiu um fluxo de controle estrito, garantindo que o tempo de alocação prévia não interferisse no tempo de busca algorítmica. O fluxo consistiu em:

1. **Geração:** Instanciação do grafo sintético em tempo de execução utilizando o gerador descrito na Seção 3, respeitando os parâmetros de n , m e topologia da iteração atual.

2. **Construção da Estrutura:** Transcrição das arestas geradas para a estrutura de dados em teste (*Forward Star* ou Lista de Adjacência).
3. **Busca do Caminho:** Execução isolada do algoritmo de Fleury, acoplado ao método de busca de pontes correspondente à iteração.
4. **Registro:** Coleta e exportação das métricas de desempenho.

Como mecanismo de segurança contra o alto custo de complexidade dos algoritmos de detecção de pontes nos grafos com 100.000 vértices, foi estabelecido um tempo limite estrito de 60 minutos por execução. Casos que excederam esse limite tiveram sua execução abortada e seus progressos foram registrados.

5.3. Métricas de Avaliação

As métricas coletadas incluíram o tempo de geração do grafo, o tempo de processamento efetivo do algoritmo de Fleury, tamanho do caminho encontrado (esperado como $m + 1$ para grafos eulerianos/semi) e o tipo euleriano validado ao final do processamento.

6. Resultados

Tabela 1. Tempo médio de execução do algoritmo de Fleury de grafos por cada tipo de conectividade (tempo em segundos).

$ V $	Algoritmo	Representação	Conexo	Semi-Euleriano	Euleriano
100	Naive Local	Adjacency List	0,010	0,006	0,021
100	Naive Local	Forward Star	0,005	0,006	0,006
100	Naive Global	Adjacency List	0,659	0,650	0,573
100	Naive Global	Forward Star	0,245	0,277	0,260
100	Tarjan	Adjacency List	0,134	0,207	0,170
100	Tarjan	Forward Star	0,103	0,192	0,196
1.000	Naive Local	Adjacency List	0,331	0,616	0,582
1.000	Naive Local	Forward Star	0,437	0,726	0,749
1.000	Naive Global	Adjacency List	745,149	1189,591	1115,868
1.000	Naive Global	Forward Star	411,810	463,066	499,416
1.000	Tarjan	Adjacency List	0,874	1,424	1,480
1.000	Tarjan	Forward Star	0,817	1,223	1,206
10.000	Naive Local	Adjacency List	22,963	69,731	78,055
10.000	Naive Local	Forward Star	24,139	84,587	85,056
10.000	Naive Global	Adjacency List	0,000	timeout	timeout
10.000	Naive Global	Forward Star	0,000	timeout	timeout
10.000	Tarjan	Adjacency List	58,681	169,980	173,697
10.000	Tarjan	Forward Star	40,743	134,952	134,322
100.000	Naive Local	Adjacency List	1130,347	timeout	timeout
100.000	Naive Local	Forward Star	1214,178	timeout	timeout
100.000	Naive Global	Adjacency List	0,000	timeout	timeout
100.000	Naive Global	Forward Star	0,000	timeout	timeout
100.000	Tarjan	Adjacency List	1126,742	timeout	timeout
100.000	Tarjan	Forward Star	1698,749	timeout	timeout

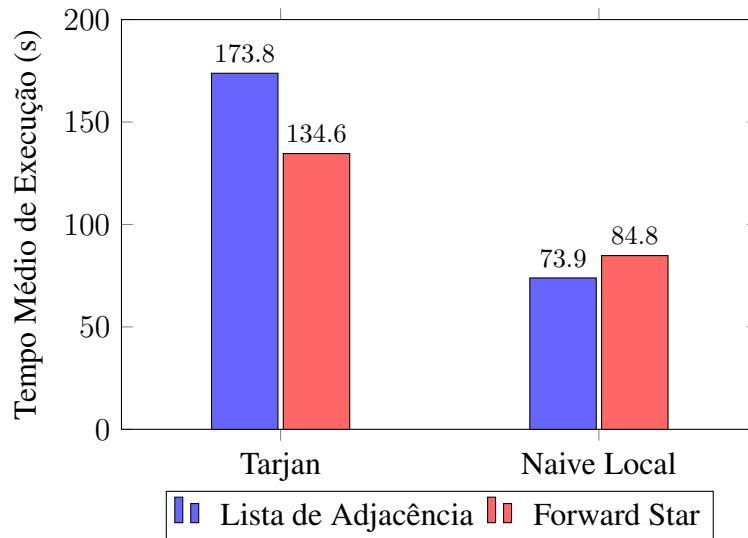


Figura 1. Comparação do impacto das representações de memória para grafos médios ($|V| = 10.000$).

Os experimentos foram conduzidos sobre as instâncias detalhadas na Metodologia. O tempo limite estrito de 60 minutos revelou com clareza os gargalos de escalabilidade, especialmente para o método *Naive Global*. Em instâncias de $|V| = 1.000$, conforme a Tabela 1, o método demonstrou sua extrema ineficiência, levando em média 499 segundos utilizando *Forward Star* e 1.116 segundos utilizando Lista de Adjacência. Para instâncias a partir de $|V| = 10.000$, o *Naive Global* excedeu o tempo limite em todas as execuções, atestando a inviabilidade computacional de remapear todos os componentes conexos a cada "passo" do caminho euleriano.

Por outro lado, o Método de Tarjan e a variação *Naive Local* conseguiram completar o caminho euleriano nas instâncias de até $|V| = 10.000$, mas atingiram o limite de tempo nas instâncias massivas de $|V| = 100.000$, completando em média 7,79% e 20,61% do caminho, respectivamente. Isso evidencia que, apesar de sua simplicidade intrínseca, o método *Naive Local*, quando executado sob demanda, obteve um desempenho substancialmente superior ao clássico método de Tarjan.

6.1. Comparação Representações de Memória

A Figura 1 apresenta um recorte das execuções para grafos com $|V| = 10.000$, ilustrando como a estrutura de dados adotada impactou as duas principais estratégias de busca.

Os dados empíricos validam enfaticamente o contraste teórico formulado na Seção 4.1. Para o algoritmo de Tarjan, que por natureza exige a exploração integral do grafo utilizando uma DFS, a representação *Forward Star* reduziu o tempo médio de execução em 21,98%. Essa aceleração é resultado direto da localidade de cache (*cache-hits*) proporcionada pelos vetores contíguos de memória.

Contudo, para a abordagem *Naive Local*, a representação de Lista de Adjacência performou 12,85% melhor em grafos médios. Isso indica que, para este cenário, o custo de instanciar um novo *Builder* para recriar o *Forward Star* a cada remoção de aresta superou o custo da sobrecarga de objetos da Lista de Adjacência.

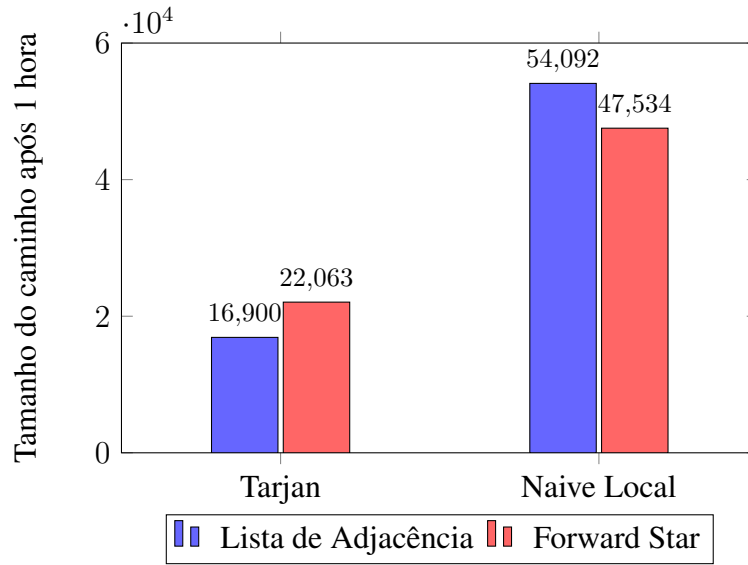


Figura 2. Comparação do impacto das representações de memória para grafos grandes ($|V| = 100.000$).

No entanto, em instâncias de grande porte ($|V| = 100.000$), conforme evidenciado pela Figura 2, o cenário se inverte. Devido ao custo para verificar se uma aresta é ponte, que cresce em $\mathcal{O}(E)$, a velocidade de iteração do *Forward Star* compensou as recriações estruturais, permitindo avançar mais no caminho euleriano antes do limite de tempo.

6.2. Crescimento do Custo Computacional (Escalabilidade)

Para avaliar a viabilidade dos algoritmos frente ao aumento exponencial do tamanho da entrada, traçou-se um perfil de escalabilidade nos cenários de sucesso, utilizando eixos em escala logarítmica (Figura 3).

Conforme ilustrado, todas as abordagens formaram progressões com inclinações semelhantes. Devido ao fato do algoritmo de Fleury executar a verificação de conectividade iterativamente para o consumo do conjunto E de arestas, tanto o método de Tarjan quanto o *Naive Local* assumiram uma característica de custo temporal prático próximo a $\mathcal{O}(E^2)$, o que explica o salto da ordem de 10^3 para 10^5 milissegundos.

Ainda assim, o *Naive Local* manteve uma superioridade absoluta frente ao Tarjan em todos os tamanhos de instâncias. No ápice do teste ($|V| = 100.000$, $|E| = 250.000$), dentro do intervalo de 1 hora, o *Naive Local* conseguiu processar em média 50.813 arestas, enquanto o Tarjan processou apenas 19.497 arestas. Essa diferença tangibiliza uma capacidade de processamento iterativo 160,62% superior a favor da abordagem local.

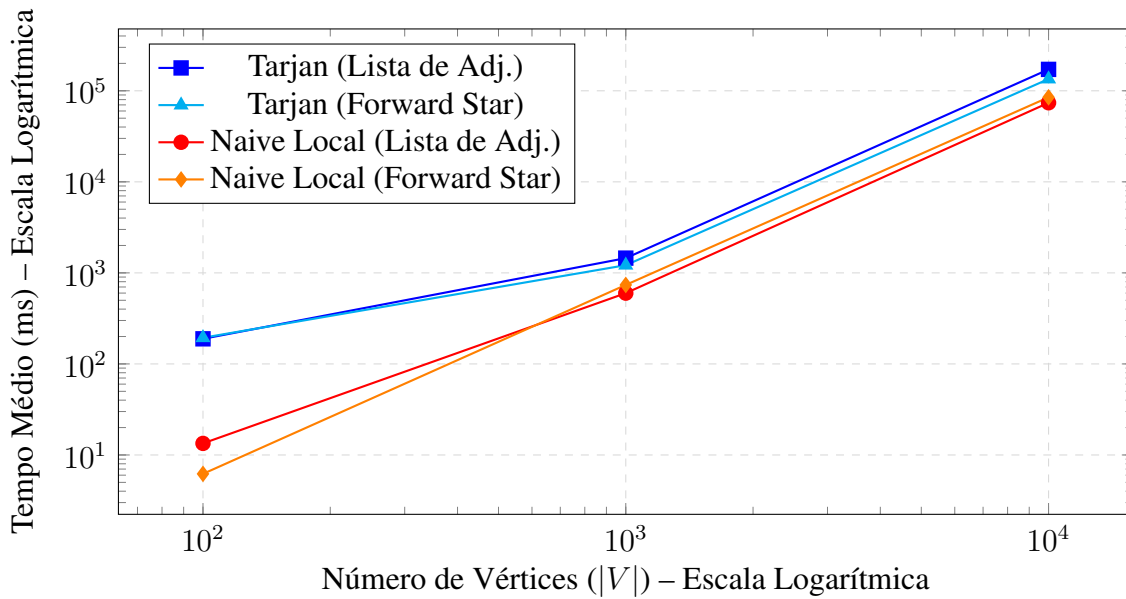


Figura 3. Crescimento logarítmico do tempo de execução em relação ao tamanho da entrada.

7. Conclusão

Este trabalho apresentou uma análise empírica detalhada do impacto do custo de identificação de pontes no desempenho do algoritmo de Fleury para a descoberta de caminhos eulerianos. Por meio de um robusto e controlado ambiente de execuções, comparou-se a eficiência da Busca em Profundidade (Método de Tarjan) em contraposição ao uso integrado da estrutura *Union-Find* sob abordagens ingênuas (*Naive Global* e *Local*), variando-se também as representações de memória subjacentes.

No que tange às representações estruturais, os resultados confirmaram que a Lista de Adjacência oferece vantagens limitadas, restritas a grafos de médio porte e a algoritmos dependentes de mutações frequentes. No entanto, para iterações massivas de leitura exigidas pela checagem de conectividade, a representação estática *Forward Star* provou ser fundamental, reduzindo drasticamente o tempo de resposta através de uma melhor localidade de cache e mitigação do *Garbage Collector*.

O principal achado desta análise reside na quebra de um paradigma clássico da teoria de algoritmos aplicados. Embora o método de Tarjan seja inegavelmente mais sofisticado e assintoticamente mais eficiente para encontrar **todas** as pontes de um grafo simultaneamente $\mathcal{O}(V + E)$, a sua integração a um método construtivo cíclico como o de Fleury gera um *overhead* proibitivo, obrigando a reexploração topológica de todo o componente conexo a cada aresta consumida.

Em contrapartida, a simplicidade do método *Naive Local*, amparada pela rapidez quase constante da estrutura *Union-Find* otimizada com compressão de caminho e união por posto, foca em validar exclusivamente se a aresta adjacente candidata rompe a conectividade. Essa estratégia de validação sob demanda, associada ao encerramento precoce, demonstrou ser significativamente superior na prática, poupando execuções desnecessárias.

Conclui-se, portanto, que verificações de pontes sob demanda e de forma isolada são substancialmente mais indicadas do que mapeamentos globais sofisticados para a montagem em tempo real de caminhos eulerianos em grafos de grande escala.

Referências

- Biggs, N. L., Lloyd, E. K., and Wilson, R. J. (1976). *Graph Theory, 1736–1936*. Clarendon Press, Oxford.
- Tarjan, R. (1972). Depth-first search and linear graph algorithms. *SIAM Journal on Computing*, 1(2):146–160.
- Tarjan, R. E. (1974). A note on finding the bridges of a graph. *Information Processing Letters*, 2(6):160–161.
- Veblen, O. (1912). An application of modular equations in analysis situs. *Annals of Mathematics*, 14(1/4):86–94.